

Choose a design pattern for your agentic AI system

Last reviewed 2026-05-28 UTC

This document provides guidance to help you choose a design pattern for your agentic AI system. *Agent design patterns* are common architectural approaches to build agentic applications. An agent design pattern offers a distinct framework for organizing a system's components, integrating the model, and orchestrating a single agent or multiple agents to accomplish a workflow.

AI agents (/docs/generative-ai/glossary#ai-agents) are effective for applications that solve open-ended problems, which might require autonomous decision-making and complex multi-step workflow management. Agents excel at solving problems in real-time by using external data and they excel at automating knowledge-intensive tasks. AI agents are suitable when you need AI to complete goal-focused tasks with some degree of autonomy. For other use cases, you can use assistive and generative AI applications. To learn about the differences between AI agents and non-agentic AI applications, see What is the difference between AI agents, AI assistants, and bots?

(<https://cloud.google.com/discover/what-are-ai-agents#what-is-the-difference-between-ai-agents-ai-assistants-and-bots>)

This guide assumes that you have a foundational knowledge of agentic AI systems and how their architecture differs from that of non-agentic systems, such as those that use direct model reasoning or retrieval-augmented generation (RAG).

(/docs/generative-ai/glossary#retrieval-augmented-generation).

For a summary of the agent pattern guidance, see the compare design patterns (#compare-design-patterns) section later in this document.

Overview of the design process

The following are the high-level steps to choose a design pattern for your agentic AI system. These steps are described in detail later in this document.

1. **Define your requirements:** Assess the characteristics of your workload, including task complexity, latency and performance expectations, cost budget, and the need for human involvement.

2. **Review the common agent design patterns:** Learn about the common design patterns in this guide, which include both single-agent systems and multi-agent systems.
3. **Select a pattern:** Select the appropriate [design pattern](#) (#compare-design-patterns) based on your workload characteristics.

This process isn't a one-time decision. You should periodically revisit these steps to refine your architecture as your workload characteristics change, your requirements evolve, or new Google Cloud features become available.

Define your requirements

The questions that follow aren't exhaustive checklists for planning. Use these questions as a starting point to identify the primary goal of your agentic system and to select the best design pattern.

- **Task characteristics:** Can your task be completed in predefined workflow steps or is the task open-ended? Does your task need to use an AI model to orchestrate the workflow?
- **Latency and performance:** Do you need to prioritize fast or interactive responses at the cost of accuracy or high-quality responses? Or can your application tolerate a delay to achieve a more accurate or thorough result?
- **Cost:** What is your budget for inference costs? Can you support patterns that require multiple calls to the model for a single request?
- **Human involvement:** Does your task involve high-stakes decisions, safety-critical operations, or subjective approvals that require human judgment?

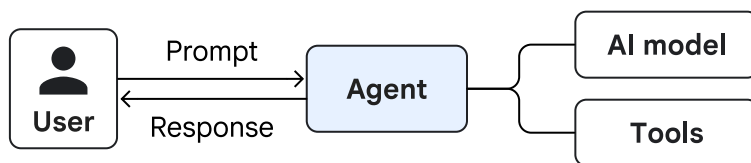
If your workload is predictable or highly structured, or if it can be executed with a single call to an AI model, it can be more cost effective to explore non-agentic solutions for your task. For example, you might not need an agentic workflow for tasks like summarizing a document, translating text, or classifying customer feedback. For information about choosing architecture components for generative AI applications that don't require an agentic infrastructure, see [Choose models and infrastructure for your generative AI application](#) (/docs/generative-ai/choose-models-infra-for-ai).

The following sections describe common agent design patterns for building a reliable and effective agentic AI system.

Single-agent system

A *single-agent system* uses an AI model, a defined set of tools, and a comprehensive system prompt to autonomously handle a user request or to complete a specific task. In this fundamental pattern, the agent relies on the model's reasoning capabilities to interpret a user's request, plan a sequence of steps, and decide which tools to use from a defined set. The system prompt shapes the agent's behavior by defining its core task, persona, and operations, and the specific conditions for using each tool.

The following diagram shows a high-level view of a single agent pattern:



A single-agent system is ideal for tasks that require multiple steps and access to external data. For example, a customer support agent must query a database to find an order status, or a research assistant needs to call APIs to summarize recent news. A non-agentic system can't perform these tasks because it can't autonomously use tools or execute a multi-step plan to synthesize a final answer.

If you're early in your agent development, we recommend that you start with a single agent. When you start your agent development with a single-agent system, you can focus on refining the core logic, prompt, and tool definitions of your agent before adding more complex architectural components.

A single agent's performance can be less effective when it uses more tools and when tasks increase in complexity. You might observe this as increased latency, incorrect tool selection or use, or a failure to complete the task. You can often mitigate these issues by refining the agent's reasoning process with techniques like the Reason and Act (ReAct) pattern (`#react-pattern`). However, if your workflow requires an agent to manage several distinct responsibilities, these techniques might not be sufficient. For these cases, consider a multi-agent system (`#multi-agent-systems`), which can improve resilience and performance by delegating specific skills to specialized agents.

Multi-agent systems

A *multi-agent system* orchestrates multiple specialized agents to solve a complex problem that a single agent can't easily manage. The core principle is to decompose a large objective into smaller sub-tasks and assign each sub-task to a dedicated agent with a

specific skill. These agents then interact through collaborative or hierarchical workflows to achieve the final goal. Multi-agent patterns provide a modular design that can improve the scalability, reliability, and maintainability of the overall system compared to a single agent with a monolithic prompt.

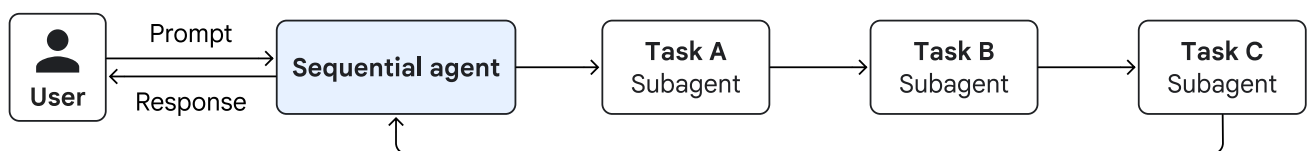
In a multi-agent system, each agent requires a specific context to perform its task effectively. Context can include documentation, historical preferences, relevant links, conversational history, or any operational constraints. The process of managing this information flow is called *context engineering*. Context engineering includes strategies such as isolating context for a specific agent, persisting information across multiple steps, or compressing large amounts of data to improve efficiency.

Building a multi-agent system requires additional evaluation, security, reliability, and cost considerations when compared to a single-agent system. For example, multi-agent systems must implement precise access controls for each specialized agent, design a robust orchestration system to ensure reliable inter-agent communication, and manage the increased operational costs from the computational overhead of running multiple agents. For an example reference architecture to build a multi-agent system, see [Multi-agent AI systems in Google Cloud](https://cloud.google.com/architecture/multiagent-ai-system) (/architecture/multiagent-ai-system).

Sequential pattern

The *multi-agent sequential pattern* executes a series of specialized agents in a predefined, linear order where the output from one agent serves as the direct input for the next agent. This pattern uses a [sequential workflow agent](https://google.github.io/adk-docs/agents/workflow-agents/sequential-agents/) (https://google.github.io/adk-docs/agents/workflow-agents/sequential-agents/) that operates on predefined logic without having to consult an AI model for the orchestration of its subagents.

The following diagram shows a high-level view of a multi-agent sequential pattern:



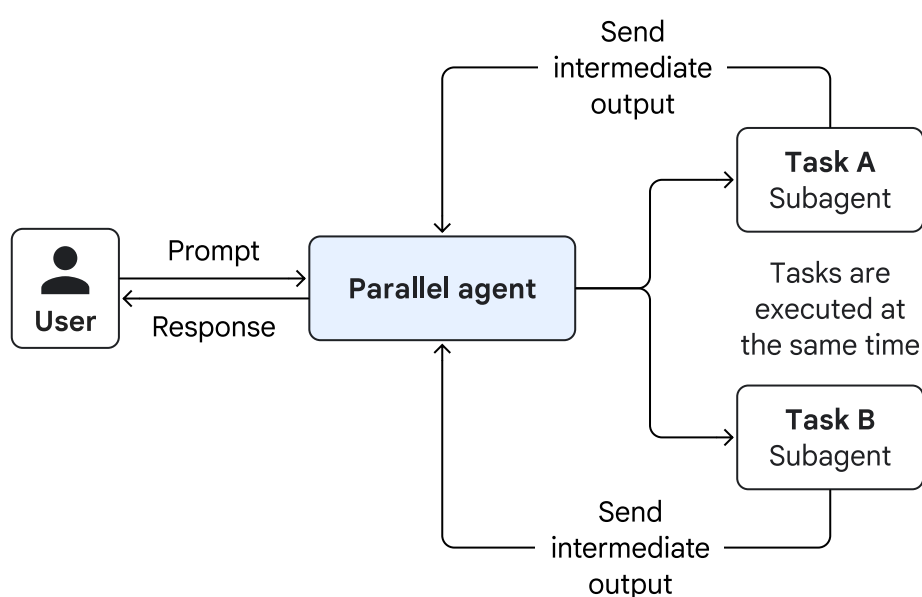
Use the sequential pattern for highly structured, repeatable processes where the sequence of operations doesn't change. For example, a data processing pipeline might use this pattern to first have a data extraction agent pull raw data, then pass that data to a data cleaning agent for formatting, which in turn passes the clean data to a data loading agent to save it in a database.

The sequential pattern can reduce latency and operational costs compared to a pattern that uses an AI model to orchestrate task workflow. However, this efficiency comes at the cost of flexibility. The rigid, predefined structure of the pipeline makes it difficult to adapt to dynamic conditions or to skip unnecessary steps, which can cause inefficient processing or lead to higher cumulative latency if an unneeded step is slow.

Parallel pattern

The *multi-agent parallel pattern*, also known as a *concurrent pattern*, multiple specialized subagents perform a task or sub-tasks independently at the same time. The outputs of the subagents are then synthesized to produce the final consolidated response. Similar to a sequential pattern (#sequential-pattern), the parallel pattern uses a parallel workflow agent (<https://google.github.io/adk-docs/agents/workflow-agents/parallel-agents/>) to manage how and when the other agents run without having to consult an AI model to orchestrate its subagents.

The following diagram shows a high-level view of a multi-agent parallel pattern:



Use the parallel pattern when sub-tasks can be executed concurrently to reduce latency or gather diverse perspectives, such as gathering data from disparate sources or evaluating several options at once. For example, to analyze customer feedback, a parallel agent might fan out a single feedback entry to four specialized agents at the same time: a sentiment analysis agent, a keyword extraction agent, a categorization agent, and an urgency detection agent. A final agent gathers these four outputs into a single, comprehensive analysis of that feedback.

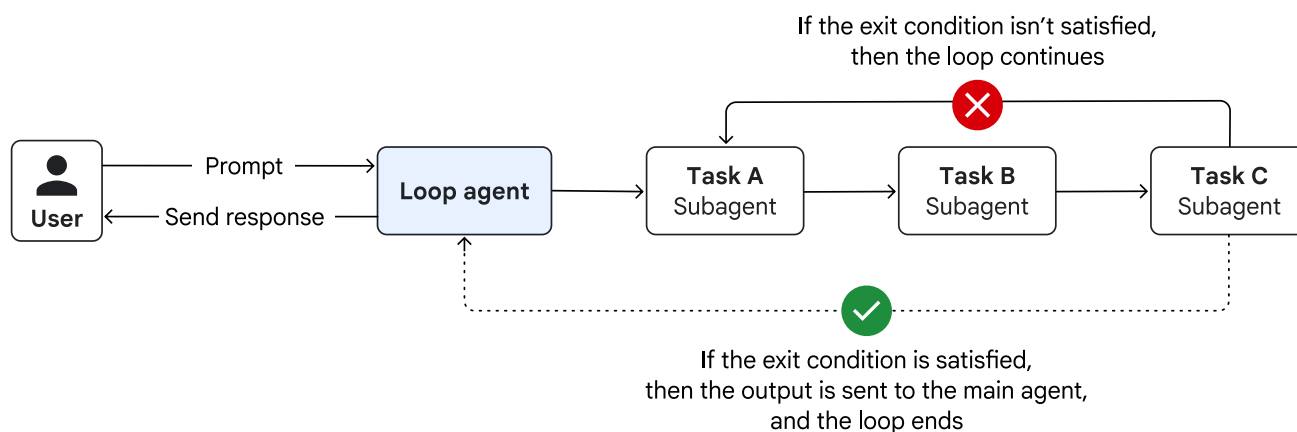
The parallel pattern can reduce overall latency compared to a sequential approach because it can gather diverse information from multiple sources at the same time. However, this

approach introduces trade-offs in cost and complexity. Running multiple agents in parallel can increase immediate resource utilization and token consumption, which leads to higher operational costs. Furthermore, the gather step requires complex logic to synthesize potentially conflicting results, which adds to the development and maintenance overhead of the system.

Loop pattern

The *multi-agent loop agent pattern* repeatedly executes a sequence of specialized subagents until a specific termination condition is met. This pattern uses a loop workflow agent (<https://google.github.io/adk-docs/agents/workflow-agents/loop-agents>) that, like other workflow agents, operates on predefined logic without consulting an AI model for orchestration. After all of the subagents complete their tasks, the loop agent evaluates whether an exit condition is met. The condition can be a maximum number of iterations or a custom state. If the exit condition isn't met, then the loop agent starts the sequence of subagents again. You can implement a loop pattern where the exit condition is evaluated at any point in the flow. Use the loop pattern for tasks that require iterative refinement (`#iterative-refinement-pattern`) or self-correction, such as generating content and having a critic agent review it until it meets a quality standard.

The following diagram shows a high-level view of a multi-agent loop pattern:



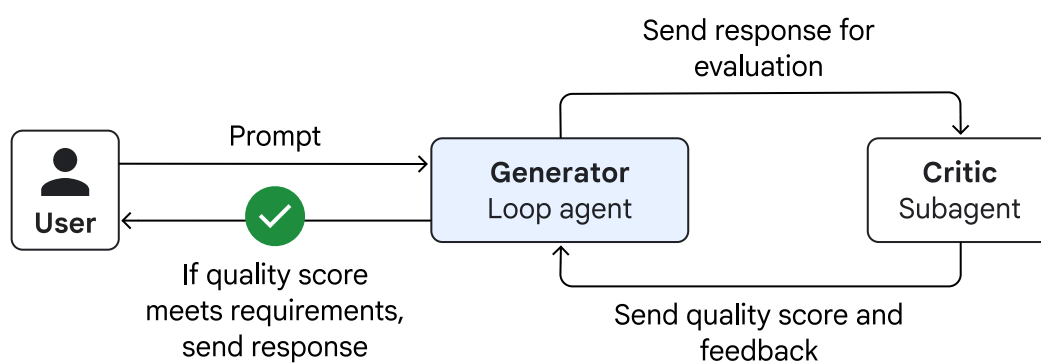
The loop agent pattern provides a way to build complex, iterative workflows. It enables agents to refine their own work and continue processing until a specific quality or state is achieved. However, this pattern's primary trade-off is the risk of an infinite loop. If the termination condition isn't correctly defined or if the subagents fail to produce the state that's required to stop, the loop can run indefinitely. This can lead to excessive operational costs, high resource consumption, and potential system hangs.

Review and critique pattern

The *multi-agent review and critique pattern*, also known as the *generator and critic pattern*, improves the quality and reliability of generated content by using two specialized agents, typically in a sequential workflow. The review and critique pattern is an implementation of the loop agent pattern (#loop-pattern).

In the review and critique pattern, a generator agent creates an initial output, such as a block of code or a summary of a document. Next, a critic agent evaluates this output against a predefined set of criteria, such as factual accuracy, adherence to formatting rules, or safety guidelines. Based on the evaluation, the critic can approve the content, reject it, or return it to the generator with feedback for revision.

The following diagram shows a high-level view of a multi-agent review and critique pattern:



This pattern is suitable for tasks where outputs must be highly accurate or must conform to strict constraints before they're presented to a user or used in a downstream process. For example, in a code generation workflow, a generator agent might write a function to fulfill a user's request. This generated code is then passed to a critic agent that acts as a security auditor. The critic agent's job is to check the code against a set of constraints, such as scanning for security vulnerabilities or verifying that it passes all of the unit tests, before the code is approved for use.

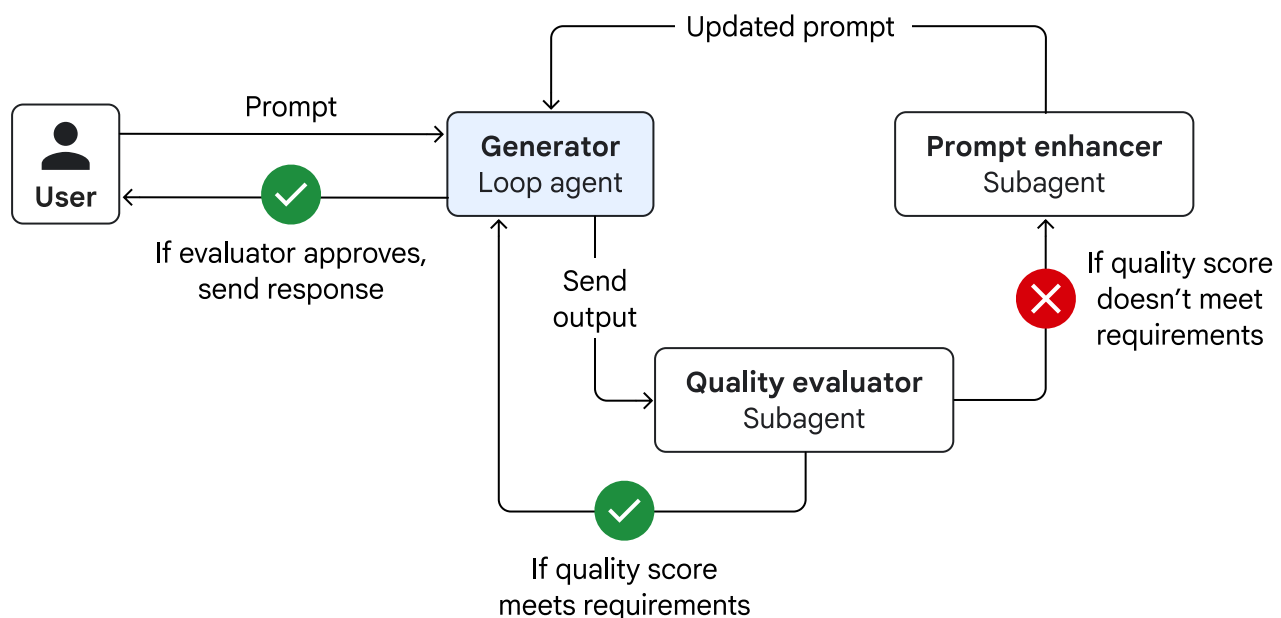
The reviewer and critique pattern can improve output quality, accuracy, and reliability because it adds a dedicated verification step. However, this quality assurance comes at the direct cost of increased latency and operational expenses. The workflow requires at least one additional model call for the critic's evaluation. If the process includes revision loops where content is sent back for refinement, then both latency and costs accumulate with each iteration.

Iterative refinement pattern

The *iterative refinement pattern* uses a looping mechanism to progressively improve an output over multiple cycles. The iterative refinement pattern is an implementation of the loop agent pattern (#loop-pattern).

In this pattern, one or more agents work within a loop to modify a result that's stored in the session state during each iteration. The process continues until the output meets a predefined quality threshold or it reaches a maximum number of iterations, which prevents infinite loops.

The following diagram shows a high-level view of a multi-agent iterative refinement pattern:



This pattern is suitable for complex generation tasks where the output is difficult to achieve in a single step. Examples of such tasks include writing and debugging a piece of code, developing a detailed multi-part plan, or drafting and revising a long-form document. For example, in a creative writing workflow, an agent might generate a draft of a blog post, critique the draft for flow and tone, and then rewrite the draft based on that critique. This process repeats in a loop until the agent's work meets a predefined quality standard or until the repetition reaches a maximum number of iterations.

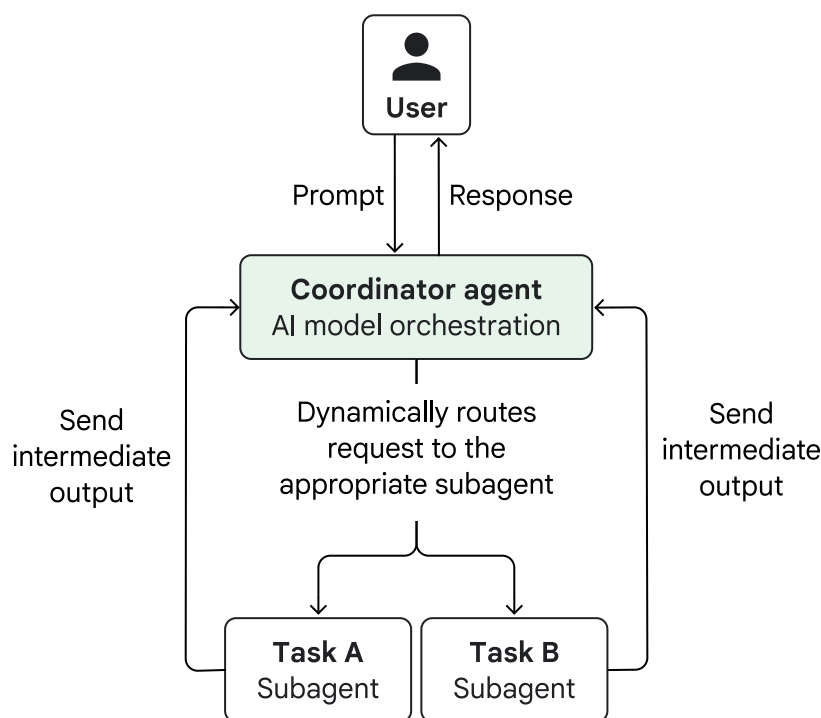
The iterative refinement pattern can produce highly complex or polished outputs that would be difficult to achieve in a single step. However, the looping mechanism directly increases latency and operational costs with each cycle. This pattern also adds architectural complexity, because it requires carefully designed exit conditions—such as a quality evaluation or a maximum iteration limit—to prevent excessive costs or uncontrolled execution.

Coordinator pattern

The *multi-agent coordinator pattern* uses a central agent, the *coordinator*, to direct a workflow. The coordinator analyzes and decomposes a user's request into sub-tasks, and then it dispatches each sub-task to a specialized agent for execution. Each specialized agent is an expert in a specific function, such as querying a database or calling an API.

A distinction of the coordinator pattern is its use of an AI model to orchestrate and dynamically route tasks. By contrast, the parallel pattern (#parallel-pattern) relies on a hardcoded workflow to dispatch tasks for simultaneous execution without the need for AI model orchestration.

The following diagram shows a high-level view of a multi-agent coordinator pattern:



Use the coordinator pattern for automating structured business processes that require adaptive routing. For example, a customer service agent can act as the coordinator. The coordinator agent analyzes the request to determine whether it's an order status request, product return, or refund request. Based on the type of request, the coordinator routes the task to the appropriate specialized agent.

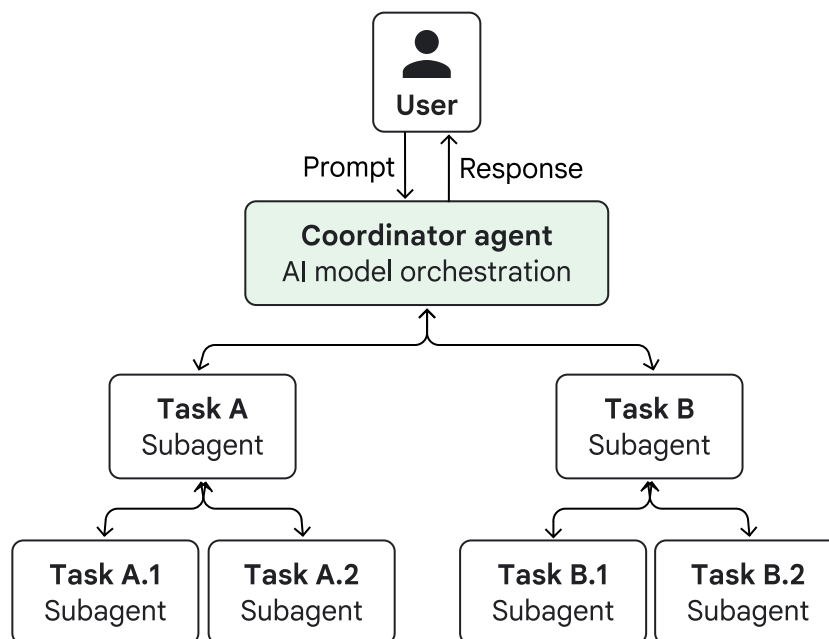
The coordinator pattern offers flexibility compared to more rigid, predefined workflows. By using a model to route tasks, the coordinator can handle a wider variety of inputs and adapt the workflow at runtime. However, this approach also introduces trade-offs. Because the coordinator and each specialized agent rely on a model for reasoning, this pattern results in more model calls than a single-agent system. Although the coordinator pattern can lead to higher-quality reasoning, it also increases token throughput, operational costs, and overall latency when compared to a single-agent system.

Hierarchical task decomposition pattern

The *multi-agent hierarchical task decomposition pattern* organizes agents into a multi-level hierarchy to solve complex problems that require extensive planning. The hierarchical task decomposition pattern is an implementation of the coordinator pattern (#coordinator-pattern).

A top-level parent, or *root*, agent receives a complex task and it's responsible for decomposing the task into several smaller, manageable sub-tasks. The root agent delegates each sub-task to a specialized subagent at a lower level. This process can repeat through multiple layers, with agents that progressively decompose their assigned tasks until the tasks are simple enough for a worker agent at the lowest level to execute directly.

The following diagram shows a high-level view of a multi-agent hierarchical task decomposition pattern:



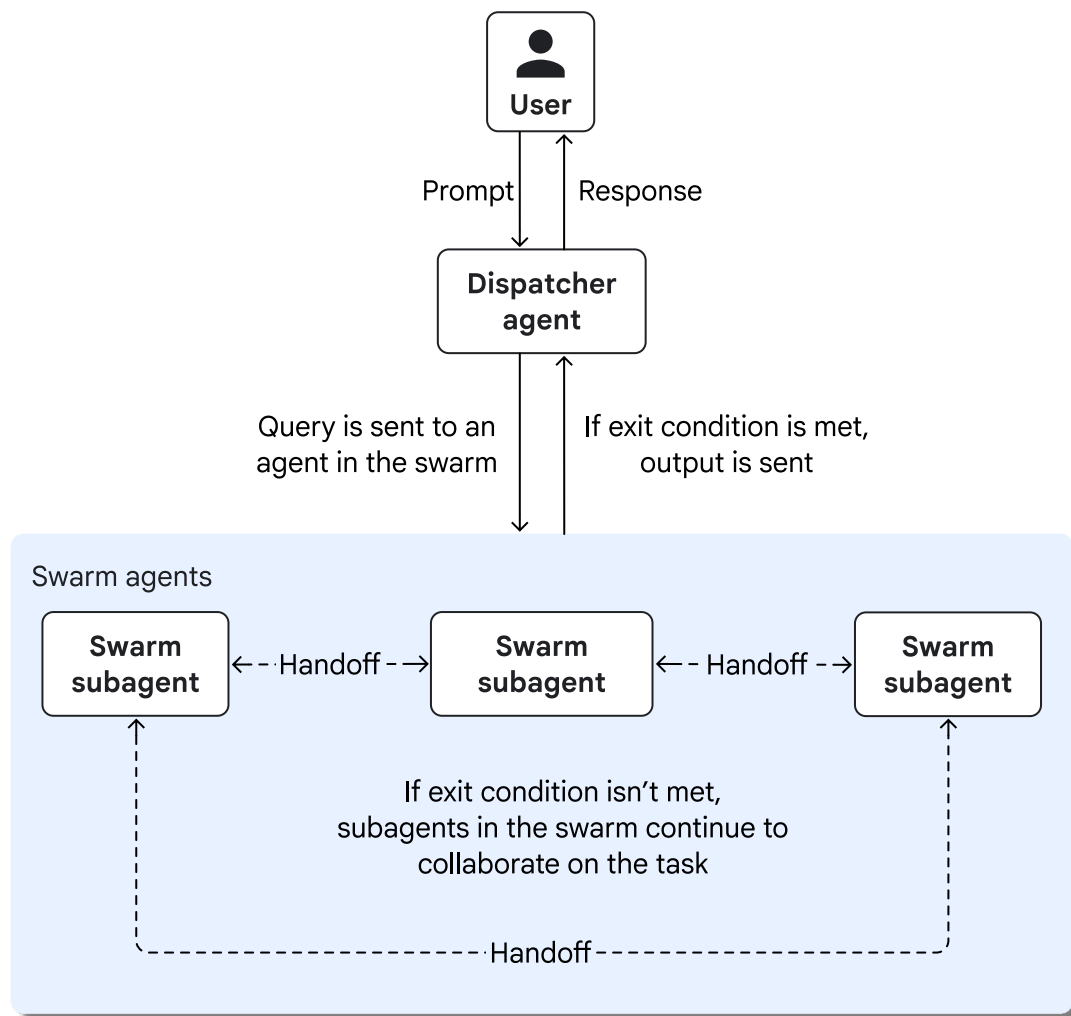
Use the hierarchical task decomposition pattern for ambiguous, open-ended problems that require multi-step reasoning, such as tasks that involve research, planning, and synthesis. For example, to complete a complex research project, a coordinator agent decomposes the high-level goal into multiple tasks such as gathering information, analyzing the findings, and synthesizing the final report. The coordinator agent then delegates those tasks to specialized subagents, such as an agent for data gathering, an analysis agent, and an agent that writes reports, to execute or further decompose.

The hierarchical task decomposition pattern is ideal for solving highly complex and ambiguous problems because it systematically decomposes them into manageable sub-tasks. This pattern can result in more comprehensive and higher-quality results than simpler patterns. However, this advanced capability introduces significant trade-offs. The multi-level structure adds considerable architectural complexity, which makes the system more difficult to design, debug, and maintain. The multiple layers of delegation and reasoning also result in a high number of model calls, which significantly increases both overall latency and operational costs compared to other patterns.

Swarm pattern

The *multi-agent swarm pattern* uses a collaborative, all-to-all communication approach. In this pattern, multiple specialized agents work together to iteratively refine a solution to a complex problem.

The following diagram shows a high-level view of a multi-agent swarm pattern:



The swarm pattern uses a dispatcher agent to route a user request to a collaborative group of specialized agents. The dispatcher agent interprets the request and it determines which agent in the swarm is best suited to begin the task. In this pattern, each agent can communicate with every other agent, which allows them to share findings, critique proposals, and build upon each other's work to iteratively refine a solution. Any agent in the swarm can hand off the task to another agent that it determines is better suited to handle the next step, or it can communicate the final response back to the user through the coordinator agent.

A swarm typically lacks a central supervisor or coordinator agent to keep the process on track. The dispatcher agent doesn't orchestrate the agentic workflow, unlike the coordinator pattern (#coordinator-pattern). Instead, the dispatcher agent facilitates communication between the swarm subagents and the user. To ensure that the swarm eventually stops and

returns a result, you must define an explicit exit condition. This condition is often a maximum number of iterations, a time limit, or the achievement of a specific goal, such as reaching a consensus.

Use the swarm pattern for ambiguous or highly complex problems that benefit from debate and iterative refinement. For example, designing a new product could involve a market researcher agent, an engineering agent, and a financial modeling agent. The agents would share initial ideas, debate the trade-offs between features and costs, and collectively converge on a final design specification that balances all of the competing requirements.

The swarm pattern simulates a collaborative team of experts, therefore it can produce exceptionally high-quality and creative solutions. However, it represents the most complex and costly multi-agent pattern to implement. The lack of an agent that uses an AI model to orchestrate can introduce the risk of unproductive loops or the failure to converge on a solution. You must therefore design sophisticated logic to manage the intricate inter-agent communication, control the iterative workflow, and handle the significant operational costs and latency that are associated with running a dynamic, multi-turn conversation between multiple agents.

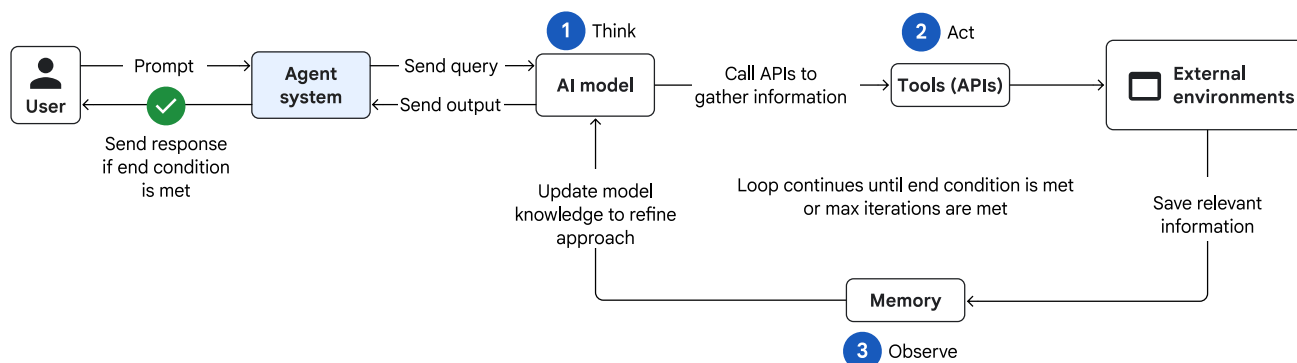
Reason and act (ReAct) pattern

The *ReAct pattern* (<https://arxiv.org/abs/2210.03629>) is an approach that uses the AI model to frame its thought processes and actions as a sequence of natural language interactions. In this pattern, the agent operates in an iterative loop of thought, action, and observation until an exit condition is met.

- **Thought:** The model reasons about the task and it decides what to do next. The model evaluates all of the information that it's gathered in order to determine whether the user's request has been fully answered.
- **Action:** Based on its thought process, the model takes one of two actions:
 - If the task isn't complete, it selects a tool and then it forms a query to gather more information.
 - If the task is complete, it formulates the final answer to send to the user, which ends the loop.
- **Observation:** The model receives the output from the tool and it saves relevant information in its memory. Because the model saves relevant output, it can build on previous observations, which helps to prevent the model from repeating itself or losing context.

The iterative loop terminates when the agent finds a conclusive answer, reaches a preset maximum number of iterations, or encounters an error that prevents it from continuing. This iterative loop lets the agent dynamically build a plan, gather evidence, and adjust its approach as it works toward a final answer.

The following diagram shows a high-level view of the ReAct pattern:



Use the ReAct pattern for complex, dynamic tasks that require continuous planning and adaptation. For example, consider a robotics agent that must generate a path to transition from an initial state to a goal state:

- **Thought:** The model reasons about the optimal path to transition from its current state to the goal state. During the thought process, the model optimizes for metrics like time or energy.
- **Action:** The model executes the next step in its plan by moving along a calculated path segment.
- **Observation:** The model observes and saves the new state of the environment. The model saves its new position and any changes to the environment that it perceives.

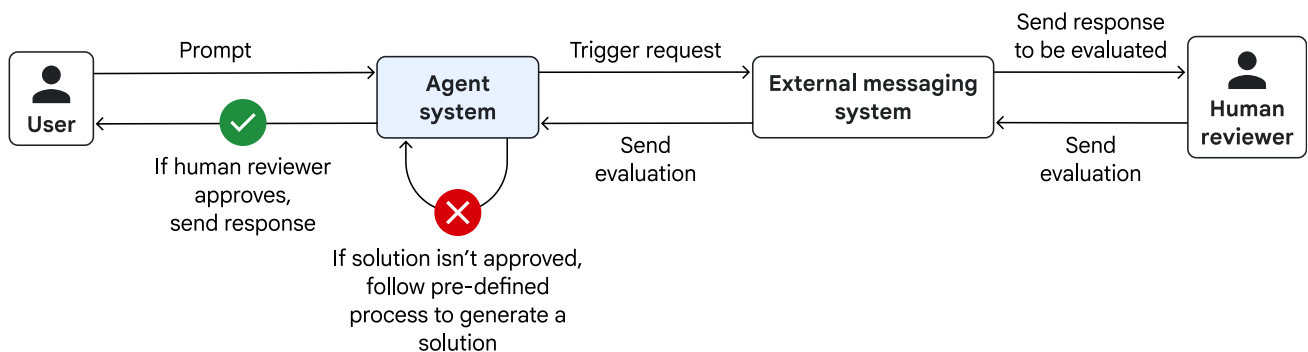
This loop allows the agent to adhere to dynamic constraints, such as avoiding new obstacles or following traffic regulations, by constantly updating its plan based on new observations. The agent continues through its iterative loop until it reaches its goal or encounters an error.

A single ReAct agent can be simpler and more cost-effective to implement and maintain than a complex multi-agent system. Model thinking provides a transcript of the model's reasoning, which helps with debugging. However, this flexibility introduces trade-offs. The iterative, multi-step nature of the loop can lead to higher end-to-end latency compared to a single query. Furthermore, the agent's effectiveness is highly dependent on the quality of the AI model's reasoning. Therefore, an error or a misleading result from a tool in one observation step can propagate and cause the final answer to be incorrect.

Human-in-the-loop pattern

The human-in-the-loop pattern integrates points for human intervention directly into an agent's workflow. At a predefined checkpoint, the agent pauses its execution and calls an external system to wait for a person to review its work. This pattern lets a person approve a decision, correct an error, or provide necessary input before the agent can continue.

The following diagram shows a high-level view of a human-in-the-loop pattern:



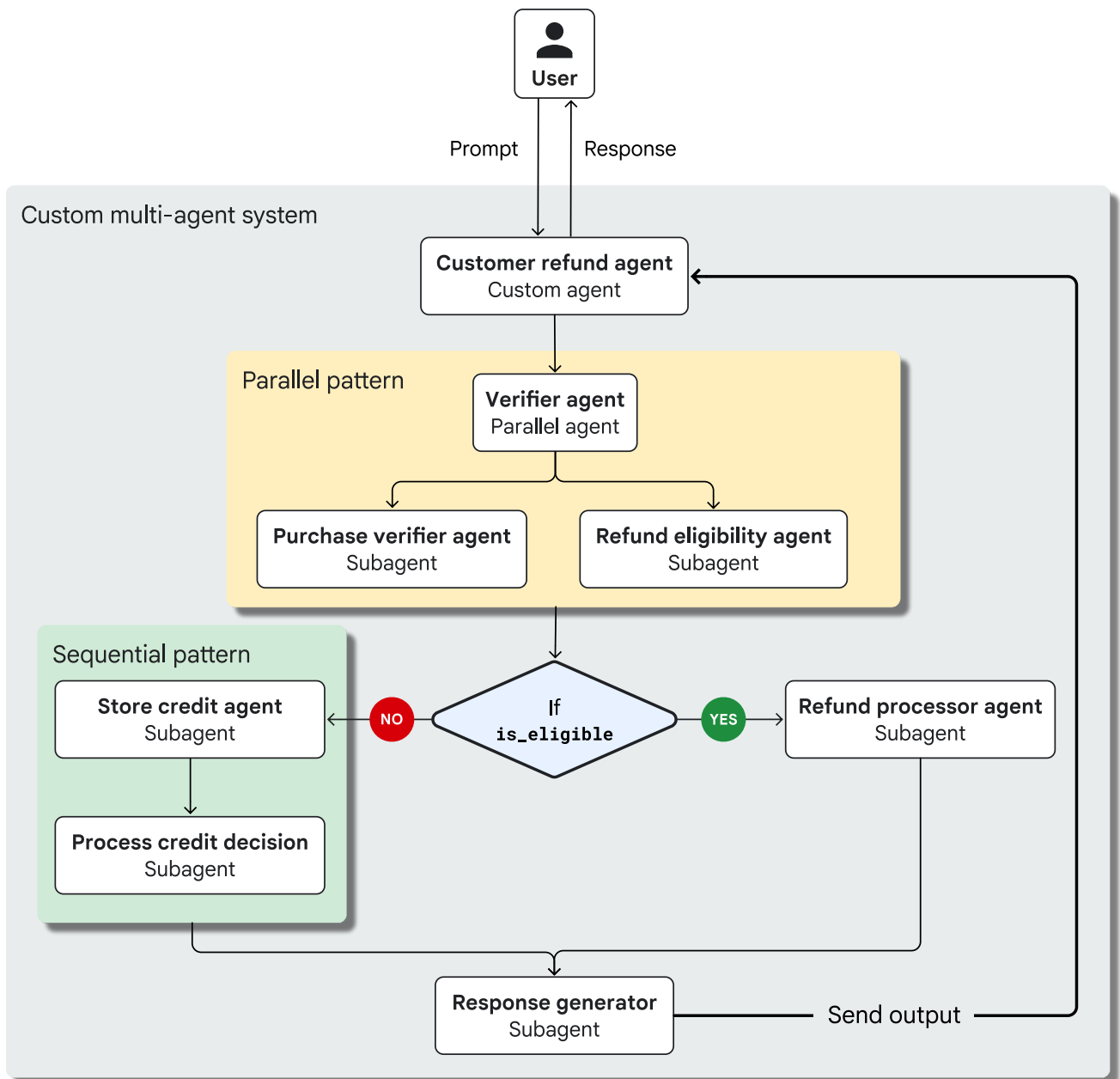
Use the human-in-the-loop pattern for tasks that require human oversight, subjective judgment, or final approval for critical actions. Such actions include approving a large financial transaction, validating the summary of a sensitive document, or providing subjective feedback on generated creative content. For example, an agent might be tasked with anonymizing a patient dataset for research. The agent would automatically identify and redact all protected health information, but it would pause at a final checkpoint. It would then wait for a human compliance officer to manually validate the dataset and approve its release, which helps to ensure that no sensitive data is exposed.

The human-in-the-loop pattern improves safety and reliability by inserting human judgment into critical decision points within the workflow. This pattern can add significant architectural complexity because it requires you to build and maintain the external system for user interaction.

Custom logic pattern

The custom logic pattern provides the maximum flexibility in your workflow design. This approach lets you implement specific orchestration logic that uses code, such as conditional statements, to create complex workflows with multiple branching paths.

The following diagram illustrates an example use of a custom logic pattern to capture a refund process:



In the preceding diagram, the following is the agentic workflow for the example customer refund agent:

1. The user sends a query to the customer refund agent that acts as a coordinator agent.
2. The coordinator's custom logic first invokes a parallel verifier agent, which simultaneously dispatches two subagents: the purchaser verifier agent and the refund eligibility agent.
3. After the results are gathered, the coordinator agent executes a tool to check whether the request is eligible for a refund.
 - a. If the user is eligible, then the coordinator routes the task to a refund processor agent, which calls the `process_refund` tool.

- b. If the user isn't eligible, then the coordinator routes the task to a separate sequential flow, starting with the store credit agent and the process credit decision agent.
4. The result from whichever path is taken is sent to the final response agent to formulate the answer for the user.

The customer refund agent example requires a unique solution for its logic-level orchestration, which goes beyond the structured approaches that other patterns offer. This workflow mixes patterns because it runs a parallel check, and then it executes a custom conditional branch that routes to two entirely different downstream processes. This type of complex, mixed-pattern workflow is the ideal use case for the custom logic pattern.

Use the custom logic pattern when you need fine-grained control over the agent's execution or when your workflow doesn't fit one of the other patterns that's described in this document. However, this approach increases development and maintenance complexity. You are responsible for designing, implementing, and debugging the entire orchestration flow, which requires more development effort and can be more error-prone than using a predefined pattern that is supported by a tool like Agent Development Kit (ADK). (<https://google.github.io/adk-docs/>).

For information about custom agents and how to implement custom logic using ADK, see Custom agents (<https://google.github.io/adk-docs/agents/custom-agents/>).

Compare design patterns

Choosing an agent pattern is a fundamental architectural decision. Each pattern offers different trade-offs in flexibility, complexity, and performance. To determine the appropriate pattern for your workload, consider the design patterns in the following sections.

Workflows that are deterministic

Workflow that are deterministic include tasks that are predictable and sequential, and that have a clearly defined workflow path from start to finish. The steps in your tasks are known in advance, and the process doesn't change much from one run to the next. The following are agent design patterns for workflows that are deterministic:

Workload characteristics	Agent design pattern
--------------------------	----------------------

• Multi-step tasks that follow a predefined, rigid workflow. • Doesn't require model orchestration. • Fixed sequence of operations. The output of one agent is the direct input of the next agent in the sequence.	<u>Multi-agent sequential pattern</u> (#sequential-pattern)
• Independent tasks that can be executed at the same time. • Doesn't require model orchestration. • Reduces overall latency by running sub-tasks simultaneously.	<u>Multi-agent parallel pattern</u> (#parallel-pattern)
• Open-ended or complex generation tasks that are difficult to complete in a single attempt. • Requires the agent to progressively improve the output over multiple cycles. • Doesn't require model orchestration. • Prioritizes output quality over latency.	<u>Multi-agent iterative refinement pattern</u> (#iterative-refinement-pattern)

Workflows that require dynamic orchestration

Workflows that require dynamic orchestration include complex problems where the agents must determine the best way to proceed. The agentic AI system needs to dynamically plan, delegate, and coordinate tasks without a predefined script. The following are agent design patterns for workflows that require autonomous and dynamic orchestration:

Workload characteristics	Agent design pattern
• Structured and multi-step tasks that require the use of external tools. • Requires fast development for a prototype of a solution as a proof of concept.	<u>Single agent pattern</u> (#single-agent-system)
• Requires dynamic routing to an appropriate specialized subagent for structured tasks with varied input. • High latency due to multiple calls to the coordinator AI model so that it can direct tasks to the appropriate subagent. • Can incur high cost due to multiple calls to the coordinator agent.	<u>Multi-agent coordinator pattern</u> (#coordinator-pattern)

- Requires multi-level model orchestration for complex, open-ended, and ambiguous tasks. - Requires comprehensive, high-quality results where decomposing ambiguity is the primary challenge. - High latency due to nested, multi-level decomposition that leads to multiple calls to the AI model for reasoning.	<u>Multi-agent hierarchical task decomposition pattern</u> (#hierarchical-task-decomposition-pattern)
- Requires collaborative debate and iterative refinement from multiple specialized agents for highly complex, open-ended, or ambiguous tasks. - Prioritizes the synthesis of multiple perspectives to create a comprehensive or creative solution. - High latency and operational costs due to dynamic, all-to-all communication between agents.	<u>Multi-agent swarm pattern</u> (#swarm-pattern)

Workflows that involve iteration

Workflows that involve iteration include tasks where the final output is achieved through cycles of refinement, feedback, and improvement. The following are agent design patterns for workflows that involve iteration:

Workload characteristics	Agent design pattern
- Requires an agent to iteratively reason, act, and observe to build or adapt a plan for complex, open-ended, and dynamic tasks. - Prioritizes a more accurate and thorough result over latency.	<u>ReAct pattern</u> (#react-pattern)
- Requires monitoring or polling tasks that repeat a predefined action, such as automated checks, until the agent meets an exit condition. - Unpredictable or long-running latency while waiting for an exit condition to be met.	<u>Multi-agent loop pattern</u> (#loop-pattern)
Tasks require a distinct validation step before completion.	<u>Multi-agent review and critique pattern</u> (#review-critique-pattern)
- Open-ended or complex generation tasks that are difficult to complete in a single attempt. - Requires the agent to progressively improve the output over multiple cycles. - Doesn't require model orchestration.	<u>Multi-agent iterative refinement pattern</u> (#iterative-refinement-pattern)

- Prioritizes output quality over latency.

Workflows that have special requirements

Workflows that have special requirements include tasks that don't follow the common agentic patterns. Your tasks can include unique business logic or they can require human judgment and intervention at critical points. Your agentic AI system is a custom-built machine designed for a single, specific purpose. The following are agent design patterns for workflows that have special requirements:

Workload characteristics	Agent design pattern
• Requires human supervision due to high-stakes or subjective tasks that might include safety, reliability, and compliance requirements.	Human-in-the-loop pattern (#human-in-the-loop-pattern)
• Complex, branching logic that goes beyond a direct linear sequence. • Requires maximum control to mix predefined rules with model reasoning.	Custom logic pattern (#custom-logic-pattern)
• Requires fine-grained process control for a workflow that doesn't fit a standard template.	

What's next

- Learn more about how to construct and manage [multi-agent systems using ADK primitives](https://google.github.io/adk-docs/agents/multi-agents/) (https://google.github.io/adk-docs/agents/multi-agents/).
- Learn how to [host AI apps and agents on Cloud Run](/run/docs/ai-agents) (/run/docs/ai-agents).
- Learn more about [Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems](https://www.amazon.com/Agentic-Design-Patterns-Hands-Intelligent/dp/3032014018/) (https://www.amazon.com/Agentic-Design-Patterns-Hands-Intelligent/dp/3032014018/).
- Learn how to [build an agent with ADK](/agent-builder/agent-engine/quickstart-adk) (/agent-builder/agent-engine/quickstart-adk).
- Learn more about how to build [multi-agent AI systems in Google Cloud](/architecture/multiagent-ai-system) (/architecture/multiagent-ai-system).
- For more reference architectures, diagrams, and best practices, explore the [Cloud Architecture Center](/architecture) (/architecture).

Contributors

Author: Samantha He (<https://www.linkedin.com/in/samantha-he-05a98173>) | Technical Writer

Other contributors:

- Abdul Saleh (<https://www.linkedin.com/in/abdulsaleh/>) | Software Engineer
- Amina Mansour (<https://www.linkedin.com/in/aminamansour/>) | Head of Cloud Platform Evaluations Team
- Amit Maraj (<https://www.linkedin.com/in/amit-maraj>) | Developer Relations Engineer
- Casey West (<https://www.linkedin.com/in/caseywest>) | Architecture Advocate, Google Cloud
- Jack Wotherspoon (<https://www.linkedin.com/in/jack-wotherspoon>) | Developer Advocate
- Joe Fernandez (<https://www.linkedin.com/in/joefernandez007/>) | Staff Technical Writer
- Joe Shirey (<https://www.linkedin.com/in/jshirey>) | Cloud Developer Relations Manager
- Karl Weinmeister (<https://www.linkedin.com/in/karlweinmeister/>) | Director of Cloud Product Developer Relations
- Kumar Dhanagopal (<https://www.linkedin.com/in/kumardhanagopal>) | Cross-Product Solution Developer
- Lisa Shen (<https://www.linkedin.com/in/lisa-shen-6167241/>) | Senior Outbound Product Manager, Google Cloud
- Mandy Grover (<https://www.linkedin.com/in/mandygrovermatc/>) | Head of Architecture Center
- Mark Lu (<https://www.linkedin.com/in/mmmarklu/>) | Technical Writer
- Megan O'Keefe (<https://www.linkedin.com/in/askmeegs>) | Developer Advocate
- Olivier Bourgeois (<https://www.linkedin.com/in/olivi-eh/>) | Developer Relations Engineer
- Shir Meir Lador (<https://www.linkedin.com/in/shirmeirlador>) | Developer Relations Engineering Manager
- Vlad Kolesnikov (<https://www.linkedin.com/in/vkolesnikov>) | Developer Relations Engineer

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the Apache 2.0 License (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the Google Developers Site

Policies (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-05-28 UTC.